Allaith Issa

Event Booking · Weekly Mini Project 02 · Team 4 **Team Leader / Frontend**

Responsibility: Frontend foundation, shared components, Confirmation screen, integration and review

You prepare what the others build on, build the Confirmation screen, and integrate the team's work. The project must be finished in two days, so build the core requirements first.

Your Flow, Phase by Phase

Phase 1: Skeleton and Team Setup

Make sure everyone can start from the same base. Tick each item when it is on main.

- [ ] Root `.gitignore` contains `node_modules`, `dist`, `*.tsbuildinfo`, `.env`; old `dist` folders are removed from git.
- [ ] `api/src` has `common/`, `events/`, `bookings/`, `types.ts`, and the two modules are registered in `app.module.ts`. `main.ts` calls `enableCors()`.
- [ ] `client/src` has `api/`, `mocks/`, `components/`, `pages/`, `types.ts`, the five routes in `App.tsx`, and `.env.example` (`VITE_API_URL`, `VITE_USE_MOCK`).
- [ ] `client/src/api/client.ts` and the three api files (`eventsApi.ts`, `bookingApi.ts`, `manageApi.ts`) are in place, and `mocks/mockData.ts` + `mocks/mockApi.ts` are copied in.
- [ ] `API_CONTRACT.md` is in the repo.
- [ ] Team branches are recreated from the updated main, and the group chat message is sent (parallel work, file ownership, branch names, do not install new libraries without telling you).
- [ ] Tell Majed and Adham the backend decision: each service keeps its own array (no StoreService), with `forwardRef` between EventsService and BookingsService. Agree on the two shared method names: one in BookingsService that returns the Active bookings of an event, one in EventsService that returns an event by id.

Phase 2: Shared Components

Small reusable pieces every screen uses. Write them in your own style (plain functions, interface for props, `export default` at the bottom).

- [ ] `LoadingMessage`, `ErrorMessage` (with Retry button) and `EmptyState`: reuse the ones from B4F Hub as they are.
- [ ] `StatusBadge`: receives a `status` (Available, Almost full, Full, Past, Active, Cancelled) and shows it with a different class per status, for example `status-badge status-full`. Use a labels object and a small function that turns "Almost full" into a valid class name.
- [ ] `Navbar`: `NavLink` to Events (`/`) and Find / Manage Booking (`/manage`), with the active link class like in B4F Hub. Place it above the routes in `App.tsx`.
- [ ] Push these first, then tell the team they can import them (Taima, Badr and Yousef start with temporary versions until then).

Phase 3: Booking Confirmation Screen (`ConfirmationPage.tsx`)

The screen shown after a successful booking.

- [ ] Read the booking from `useLocation().state`. Agree with Badr that after success he calls `navigate("/confirmation", { state: { booking } })`.
- [ ] If there is no booking (page was refreshed), show a clear message with a link to Find / Manage Booking. Never a blank page.
- [ ] Show: booking code (prominent), event title, date, time, location, customer name, phone, number of places, booking status (`StatusBadge`).
- [ ] Show the event's updated remaining places and its status, taken from `booking.event`.
- [ ] Add links: Back to events, and Find / Manage Booking.
- [ ] Test with mocks: create a booking from a temporary button, confirm remaining places dropped, then remove the temporary button.

Phase 4: Global Styling (`index.css`)

One consistent look for all screens. Plain CSS with class names, as in B4F Hub.

- [ ] Base layout: page container, navbar, headings, buttons, inputs.
- [ ] Classes for cards and lists (event cards, booking details).
- [ ] Status badge colors: one color per status (Available, Almost full, Full, Past, Active, Cancelled).
- [ ] Message styles: loading, error (`.message.error`), form error, empty state.
- [ ] Tell the team which class names exist so they reuse them instead of inventing new ones.

Phase 5: Integration and Review (ongoing, from the first pull request)

Your main role as team leader.

- [ ] Review and merge each pull request into main one at a time. After each merge, run both projects and check nothing broke.
- [ ] If a merge conflict appears (usually in `app.module.ts`, `App.tsx` or `package.json`), keep both sides and run the project again.
- [ ] Check that everyone uses the shared `types.ts`, `client.ts` and the shared components.
- [ ] When an endpoint is announced as ready, switch to the real backend (`VITE_USE_MOCK=false`) and test that screen. If the backend response differs from `API_CONTRACT.md`, tell its owner.
- [ ] Walk through the four flows end to end with the real backend: A (discover), B (book), C (event fills up), D (find and cancel).
- [ ] Run the invalid cases from section 11 of the specification and make sure each shows a clear message.
- [ ] Check the Minimum Deliverable list (section 14) item by item. Core first; bonus only if everything else works.

Phase 6: Presentation Preparation

Your speaking part in English.

- [ ] Topic: the problem, who the product is for, and the team introduction.
- [ ] Make sure every member knows their part and speaks.
- [ ] Before presenting: restart backend and frontend from scratch and check the demo data is ready.

Definition of Done

- [ ] Skeleton, contract, mocks and api files are on main and the team has started
- [ ] Shared components and Confirmation screen work with both mock and real backend
- [ ] All screens look consistent
- [ ] All four flows and the invalid cases work with the real backend
- [ ] Presentation part prepared; every member ready to speak